Cormen, Leiserson, Rivest, Stein: Introduction to Algorithms (4th edition). MIT Press (2022), ISBN: 9780262046305

Test Content
9 questions, 32 points. (7 4-point and 2 2-point questions.) It covers the whole semester, weighted **a little more towards graphs.**

To cover

- ☐ A tricky question per unit
- ☐ Summer 2025 Exam
- ☐ Tony's Piazza questions

Important takeaways
Analysis
    Week 1: Probability NOT REALLY TESTED
    Week 6: Amortized analysis. TUTORIAL
        QuickSort
        Las Vegas vs. Monte Carlo

Trees
    Week 2: Heap
        HeapSort
        BuildMaxHeap
    Week 3: Dictionary, BST
    Week 4: Balanced Trees
    Week 5: Hashing

Graphs
    Week 8: Adjacency mtx vs list
        BFS, DFS implementation
    Week 9: DFS
        DFS to find cycles. Kosaraju for SCCs
        Topological sorting
    Week 10: MSTs
        Prim's vs. Kruskal's
        Spanning tree |E| = |V|- 1.

Other

Week 11: Disjoint Sets

LL, Modified LL, Trees, Flattening

Topics

- [ ] Week 6: Amortized Analysis
  - [ ] Worksheet
- [ ] Week 9: DFS
  - [ ] Children problem
  - [ ] Kosaraju
  - [ ] SCC

Tutorials

- [ ] 11. Disjoint sets
- [ ] 10. MST
- [ ] 9. DFS, SCC
- [x] ~~8. Graphs, BFS, DFS~~
- [ ] 6. Amortized Analysis
- [ ] 5. Hashing
- [ ] 4. AVL Trees
- [ ] 3. Dictionaries, BST
- [ ] 2. Heaps
- [ ] 1. Complexity, ADT
- [ ] Exam Jam

Worksheets

- [x] ~~11. Disjoint sets~~
- [x] ~~10. MSTs~~
- [x] ~~9. DFS, SCC~~
- [x] ~~8. Graphs, BFS, DFS~~
- [x] ~~6. Amortized Analysis~~
- [ ] 5. Hashing
- [x] ~~4. AVL Trees~~
- [ ] 3. Dictionaries, BST
- [x] ~~2. Heaps~~
- [ ] 1. Complexity, ADT

Week 1: Complexity, ADT

ADT

A black box model defining a data type by behaviour without regard for internal logic or implementation.

We implement ADTs with data structures.

Complexity

Random variables, distributions, etc.

Week 2: Priority Queues and Heaps

Priority Queue ADT

Operations

- Insert(Q, x) — Add element with priority.
- ExtractMx(Q) — Remove and return max element
- Max(Q) — Return max element
- IncreasePriority(Q, x, k) — Increase x.priority to k

Worst case running times

	unsorted list	sorted list	Heap
`Insert(Q,x)`	$\Theta(1)$	$\Theta(n)$	$\Theta(\log n)$
`Max(Q)`	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
`ExtractMax(Q)`	$\Theta(n)$	$\Theta(1)$	$\Theta(\log n)$
`IncreasePriority (Q,x,k)`	$\Theta(1)$	$\Theta(n)$	$\Theta(\log n)$

Heap DS

Max-heap property: Every node has key/priority >= keys of its immediate children.
Thus every node is larger than all its descendants. Subtrees are subheaps

Complete binary tree:

1. All its levels are full except possibly bottom.
2. All nodes in bottom level are as far left as possible

Binary max-heap: A nearly complete binary tree with max heap property
Such a heap guarantees h=log n

Storing a heap: Heaps are easily stored in arrays rather than linked tree structure

Operation runtime of Heap

Operations

- Insert(Q, x) O(log n) ← Requires bubbling up
- ExtractMax(Q) Θ(log n)
- Max(Q) Θ(1) worst case
- IncreasePriority(Q, x, k) Increase x.priority to k

Implementation of insert

There is one place we can insert to keep it complete.
We append the new element x to the end of the heap;

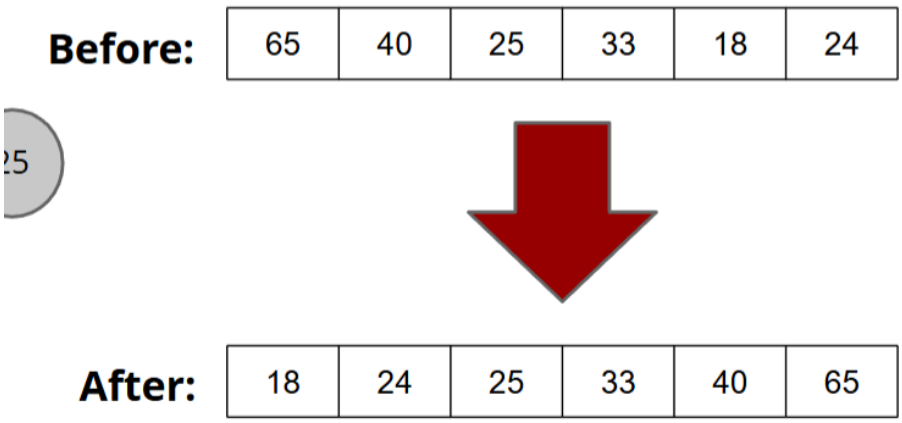
Of course, the heap property breaks. So we **bubble up**: compare with its parent
Runtime: height of tree, which is O(log n)

Implementation of ExtractMax
We can't just delete the root. We have to get rid of the final slot.
What we can do: swap the key of the root with the final leaf (1st index with nth index). To restore heap property, we **bubble down**.
O(height) + some constant work

Implementation of IncreasePriority
Simply change the key of the node and bubble up.

HeapSort

Sorts array in O(n log n) time
Problem: Given a max-heap-ordered array, how do we modify it to be a sorted array?
Solution: ExtractMax and put it at the end (bubble it down). Do this n times.



Pseudocode of HeapSort

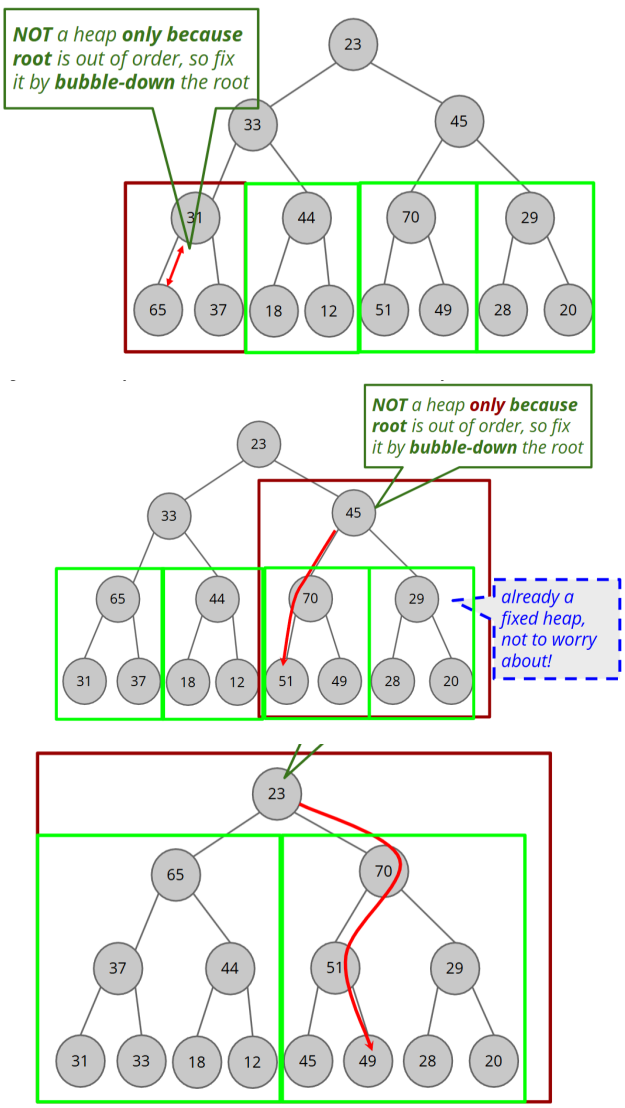
```
HeapSort(A):
    BuildMaxHeap(A):            // Need heap ordered array A
    for i = A.size down to i=2 {
        swap A[1] and A[i]
        A.size = A.size - 1
        BubbleDown(A,1)
```

BuildMaxHeap(A)

First, how do we convert an array into a max heap array in O(n)?
Approach: If the left and right subtree is heap, and only root out of order, can simply bubble it down to get a heap. So it suffices to adjust heap order from bottom up.

Take the bottom most subtree. It can be a heap if you bubble the linking one down. Then do the same for its parent and siblings

Specifically, **how**? We start from index floor(n/2)



BMH Pseudocode

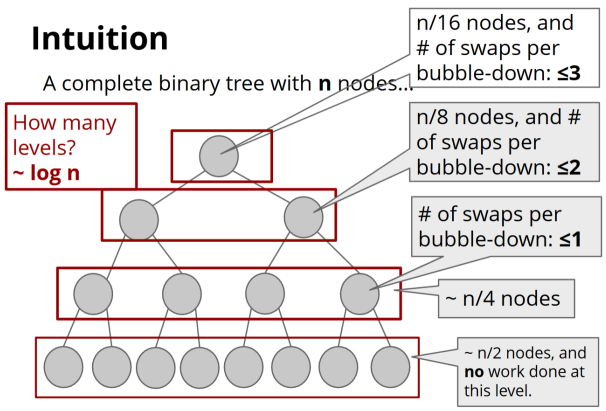
```
BuildMaxHeap(A):
    for i ← floor(n/2) downto 1:
        BubbleDown(A, i)
```

BMH Runtime

Each bubble down is O(log n). Roughly n/2 times. So should be O(n log n)? Or maybe better!

It's actually O(n)!

Analysis



So, total number of swaps

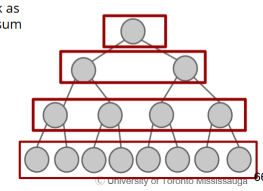
$$T(n) = 1 \cdot \frac{n}{4} + 2 \cdot \frac{n}{8} + 3 \cdot \frac{n}{16} + \dots$$

$$= \sum_{i=1}^{\log n} i \cdot \frac{n}{2^{i+1}} \quad \leq \sum_{i=1}^{+\infty} i \cdot \frac{n}{2^{i+1}}$$

$$= n \boxed{\sum_{i=1}^{+\infty} \frac{i}{2^{i+1}}} \quad = 1$$

same trick as Week 1's sum

$$= n$$



WHAT IS THAT SUM THING?

n number of swaps! O(n) compared to

Week 3: Dictionaries, BST

Dictionary ADT

Operations

1. Search(S, k)

2. Insert(S, x)
3. Delete(S, x)

Binary Search Tree DS

Need not be complete.
Property: is sorted. We use in-order traversal to obtain a sorted list
Each node contains

- key, left, right, p

Operations on a BST

- TreeSearch(root, k)	Return node with key k or NIL otherwise
- TreeMinimum(x)/TreeMaximum(x)	Return node with lowest/highest key of tree
- Successor(x) / Predecessor(x)	Find node following x; TreeMinimum(x.right)
- TreeInsert	Insert x into BST. If key exists, replace y with x
- TreeDelete	sdlfij

Implementation

TreeSearch: If root is NIL or root is k, done. Else search left, then right.

O(h) runtime (root to leaf)

TreeMin/Max: Keep going left. Or right.

O(h) runtime. O(n) worst case

Successor: Two cases: Has right child, or does not.

Case 1: Has a right child. Use TreeSearch(k). Then return TreeMinimum(x.right)

Case 2: No right child. Then go up the parent chain; if x is the right child, keep going. if x is a left child, stop; we have found the smallest guy bigger than x

```
Successor(x):
    if x.right != NIL: return TreeMinimum(x.right)
    y = x.p
    while y != NIL and x == y.right
        x = y
        y = y.p
    return y
```

Successor runtime: Also O(h)

TreeInsert: Similar to TreeSearch, go down left and right. When the next position is NIL, insert. If same, replace.

TreeDelete: 3 cases

Case 1: x has no child;	Just delete it.
Case 2: x has one child;	Connect parent to child. Child becomes child of parent.
Case 3: x has two children;	Replace parent with Successor(x) = ExtractMax(x.right)

Week 4: Balanced Trees, Augmented Data Structures

AVL Tree DS

A spin on the binary search tree. It is self-balancing. The height of one subtree must differ from the other by at most 1 level.

Each node does not store the height attribute. Only the balance factor (-1, 0, 1)

Balance Factors

Tree Rotations

Double rotations

Operation runtime

Insert:	O(h) = O(log n)
Search:	O(h)=O(log n) as in BST. No worst case

In general, the operations are the same as a BST.
If on insert, it is not balanced, need to rebalance.

Then we update the balance factor, which takes O(log n) worst case

If the balance factor of a child changes on insert (I have no children but i still insert) then that child's balance factor should change.

On recursive return, if the child's balance factor changes, then we must update our own balance factor based on whether the left or the right child changed.

That is, it goes up.

Delete: ?? Tutorial

Augmenting Data Structures

Sometimes, the textbook definition of an ADT or data structure does not help very much to solve a problem we may have.

We can augment data structures to achieve certain goals that the original data structure would not be able to achieve, or at least in an efficient time.

Practice this!

Theorem for augmenting AVL trees

THEOREM; If additional information only depends on its children and itself, then this info can be maintained efficiently during Insert and Delete without affecting their O(log n) worst-case runtime.

Week 5: Hashing

Hash table data structure

Operations from Dictionary

1. Search(S, k)
2. Insert(S, x)
3. Delete(S, x)

Hash table runs constant time for all

Resolving Collisions: Chaining

Definition: Each slot stores a linked list. Then search, insert, delete become O(length chain).

So worst case, O(n)

The average case is pretty good though.

Average Case Analysis

Assume simple uniform hashing. Every key k is equally likely to hash to any of the m buckets.

Intuitively, if we insert n keys into m slots with simple uniform hashing, the chains at every slot will have on average the same length n/m

Formally:

Pr(h(k) = j) = 1/m

Define random variable N(k) to be the number of elements examined during search for k.

Then average case running time is E[N(k)]

Computing for an unsuccessful search

E[N(k)] <= Sum Pr[h(k') = h(k)] + 1 (1 + number of elements before x in chain)

== 1 + n/m

sdfsdfjsldf proof omitting

Load factor: Let alpha = n/m. Expected number of keys per bucket.

Given O(1) steps for calculating h(k), accessing T:

Average case running time for search is O(1+alpha)

Drawback: We assumed simple uniform hashing. Can we really get this?

Ans: Yes if we choose good hashing

Resolving Collisions: Open Addressing / Probing

Summary: We choose good hash functions

Linear probing

Quadratic probing

Double hashing

Heuristics

Want-to-haves in a hash function

Note that objects are just a bit string. So any key can be associated with an integer easily.

A hash function must efficiently map these large integers to a small set of integers

What do we want?

1. Want h(k) to depend on every bit of k so that every difference is accounted for
2. h(k) spreads out values: k mod 2 vs k mod 263
3. h(k) should be efficient to compute

It is difficult to get all three. But we can try

Division method
modding

Multiplication method

Week 6: Amortized Analysis and QuickSort

Motivation

The **amortized sequence complexity** is the **mean cost per operation** over a **sequence** of operations. Like

Total worst case sequence complexity / m

Distinct from average-case analysis; there is NO probability or expectation involved. We do amortized analysis for the total complexity of a **sequence** of operations. This is unlike average case, which is interested in a **single** operation.

Aggregate Method vs. Accounting Method

Aggregate Method: Sum up the costs of operations and divide by number of operations

Case study: Dynamic arrays. Initialized with a fixed number of slots. When appending, if the array is full, then we double the array size and reallocate all the numbers.

Cost sequence
1, 2, 3, 1, 5, 1, 1, 1, 9, 1, 1, 1, …

Usually just add 1. Otherwise, we copy the (n/2) elements and append 1. So those operations are (n/2+1)

Accounting method: How much money do we need to earn at each operation, so that all future costs can be paid for?

QuickSort

We sort an array this way
Approach: Pick a pivot (the last one)

Partition the array by putting items smaller than pivot to the left, and vv for right.
Recursively partition the sub arrays now.

Worst case analysis: Here we will prove Upper and Lower bounds.
Claim 1: Each element in A can be chosen as pivot at most once
A pivot never goes into a sub array
Claim 2: Any pair (a,b) in A they are compared with each other at most once.
This is because comparison only happens when a or b is chosen as pivot. By Claim 1, that pivot will never be chosen again

The **upper bound** on the total number of comparisons is no more than the total number of pairs.
T(n) <= (nC2) = n(n-1)/2. ⇒ T(n) is in **O(n^2)**
This cleverly uses our claims. We come up with the claims intuitively!

The **lower bound.**
i.e. worst case running time bound below by cn2
⇒ Just find one terrible input!
Funny because the worst input with awful partitions (everything on one side) is choosing an already sorted array.
→ Choose A[n]. We compare it n-1 times.
→ Recurse to subarray, choose A[n-1]. n-2 comparisons.
→ n-3 and so on.
We do (n- 1) + (n-2) + … + 1
= n(n-1)/2 as above.
⇒ **Ω(n^2)**

Therefore the worst-case runtime **T(n) = Θ(n^2)**.

This doesn't seem any better than stupid bubble sort! But in the average case, it's different.

Average case analysis: We average over sample space and input distribution.
Assume all permutations of array [1, 2, …, n] are equally likely.

Let X be the random variable representing the number of comparisons performed on a sample array drawn from the sample space. Let's compute E(X)

Define indicator random variable:

Xij = 1 if values i and j are compared.

Then E(Xij) = Pr(i and j are compared) $\leftarrow$ Binomial thing

We sum over all possible pairs

Total number of comparisons = sum sum Xij.

$\Rightarrow$ E(X) = E(sum sum Xij) = sum sum E(Xij)

$\Rightarrow$ **E(X) = sum sum Pr(i, j are compared)**

How do we compute that probability?

Let Zij = i, i+1, …, j

Clever claim: i and j are compared if and only if among all elements in Zij, the first element to be picked as pivot is either i or j.

($\Rightarrow$) If i is chosen as pivot, then j must be compared to i

($\Leftarrow$) If kth element is the pivot, where k is between i and j, then i and j will be split into two partitions.

Thus Pr(i, j are compared)

= Pr(i or j is the first among Zij chosen as pivot)

= 2/(j - i + 1)

There are j-i+1 numbers in Zij, and each of them is equally likely to be chosen as first pivot

Average-case (3)

$$E[X] = \sum_{i=1}^{n} \sum_{j=i+1}^{n} \Pr(i \text{ and } j \text{ are compared})$$

$$= \sum_{i=1}^{n} \sum_{j=i+1}^{n} \frac{2}{j-i+1}$$

$$\in \mathcal{O}(n \log n)$$

Upper bounded by

$$n \sum_{k=1}^{n} \frac{1}{k}$$

Analysis Over!

Summary of QuickSort runtime

Worst case is $\Theta(n^2)$. Average case is $O(n \log n)$, given uniformly random input

Randomized Algorithms

Drawback of assumption

May not always be uniformly random input! We can address this by shuffling the input array uniformly randomly.

Randomised-QuickSort(A):

```
    permute A uniformly randomly
    QuickSort(A)
```

Las Vegas: Deterministic answer, random runtime. **Correctness**

We will always tend to the right answer. Think to numerical analysis root finding methods

Monte Carlo: Deterministic runtime, random answer. **Fixed time, approximate result.**

The above is a Las Vegas algorithm.

Week 8: Graphs

Definitions

Graph: A **graph** G is G = (V,E) where V is a set of vertices and E is a set of edges between vertices (v, w)

A graph may be **directed** or **undirected**. Vertices are **adjacent** if they have an edge between

A graph may be **weighted** or **unweighted**

Cycle: A sequence of distinct edges (v0, v1), (v1,v2) … (vn, v0). That is, starts and ends at the same vertex
Cyclic graph: A graph containing a cycle. Undirected graphs are easier to have a cycle

Path: A sequence of edges between two vertices
Length: Number of edges in a path. In the weighted case, this number is weighted.
Distance: The **length** of the shortest **path** between two vertices.

Connected: A graph is connected if for every pair of vertices there exists a path. Unconnected otherwise.

Graph ADT

- Vertices(G) Return list of all vertices V of G.
- CheckAdjacent(G, u, v) Return whether vertices u and v are adjacent in G.

Data structures for Graph ADT

We consider two options: adjacency matrices and adjacency lists.

Adjacency Matrix:

|V|x|V| matrix A, where the columns list out all vertices in V, and so do the rows.

Then, each entry is a (vi, vj) slot. We write 1 if there is an edge between the two, and 0 otherwise.

Space: $|V|^2$. This is obvious

THM: Adjacency matrix of an undirected graph is symmetric.

Adjacency list

Consider a list
Each vertex vi stores a list of all other vertices that it has an edge to. That is, each vertex vi stores a list A[i] of vj that satisfies (vi, vj) in E

Space: $|V|+|E|$
$|V|$ for the list
$|E|$ for the list of corresponding edges. There are only $|E|$ entries
Although if undirected, $|V|+2||E|$

Matrix vs. List

Matrix is $\Theta(|V|^2)$. List is $\Theta(|V|+|E|)$.

Which is more space-efficient?

Adjacency list: If the graph is not very dense, i.e. $|E|<<|V|^2$
Adjacency matrix: otherwise.

Which is more time-efficient

Adjacency matrix: constant time lookup (just check if entry is 1 or 0)

Traversal: BFS

Level-by-level traversal.

In breadth first search we use a queue.

Rough algorithm (tree)

Make a Queue. Enqueue the root. While not empty, Dequeue(x), and Enqueue each of the children of x

Algorithm

It's different for a graph. Instead of a root, we start with a source.

Also, we must avoid visiting the same place twice.

To do so, we “label” each vertex using colours: White, Gray, Black. This is done by taking the list of vertices V, assigning each White.

Then when we encounter a vertex (and it is White), colour it Gray. Once we encounter all of its neighbours (children), then it is explored and we colour it Black.

In addition to storing colours, we store

BFS Tree

After performing BFS we get a BFS tree. Also, if the graph is connected then the tree contains all vertices.

The d[v] values we stored at each vertex are very useful. They indicate the shortest path distance

Runtime analysis (using adjacency list)

Note

- We visit each vertex once

  $\Rightarrow O(|V|)$
- For each vertex, we check all of its neighbours. That is, we check its edges. Each edge is checked by each of its two end vertices. So

  $\Rightarrow O(|E|)$
- Combining, we get

  $\Rightarrow O(|V|+|E|)$

Runtime analysis (adjacency matrix)

Similar to the case with an adjacency list, we visit each vertex once. So $O(|V|)$ by default.

But we can’t just traverse all neighbours. Instead, we must traverse ALL vertices. Therefore

- Traversing neighbours becomes $O(|V|^2)$

So the whole thing becomes $O(|V|^2)$

Traversal: DFS

Rough algorithm (tree)

Almost exactly like BFS

DFS():

Make a Stack. Push the root. Pop(x) and DFS on each child

Or we can rewrite.
Take root, perform operation. For each child, DFS.

This is quite useful

Algorithm: On a graph
Like in BFS, we store extra info

pi[v]. Whose neighbour?
s[v]: The time at which the vertex is first encountered
f[v]: The time when all neighbours had been visited

The new algorithm

1. Initialize source as gray, time = time + 1, set start time.
2. For each neighbour v of u, if neighbour is white, pi[v] = u and we recurse into it.
3. Now we have returned, so we have finished the depth here. We increment time and set the vertex's finish time as now.

Runtime analysis (Adjacency list)
We have

- Visit each vertex once ⇒ O(|V|)
- Visit all its neighbours ⇒ O(|E|)

⇒ O(|V|+|E|)

Week 9: DFS

Applications of DFS

What can you do with DFS?

- Detect whether a graph has a cycle

Determining descendants/ancestors with a DFS forest
IsDescendant(y, u) # is y a descendant of u?

How? → Naive idea: trace back pi[v] pointers from y and see if you find u.
Better idea: Parenthesis

Parenthesis structure: Consider the intervals (s[v], f[v]).

Interval of u CONTAINS interval of v $\Leftrightarrow$ v is a descendant of u.

Edge types

Tree edge: An edge in the DFS-forest
Back edge: Non-tree edge pointing from a vertex to its ancestor in the DFS forest
Forward edge: A non tree edge pointing from a vertex to its descendant in the DFS forest
Cross edge: Other edges

THEOREM
A directed graph contains a cycle if and only if DFS yields a back edge

So we can compute: A cycle exists if there exists an edge from some vertex u to an ancestor v.

```
IsCycle Algorithm:
For edge (u, v) in E:
    if s[v] < s[u] < f[u] < f[v] return True; it is a descendant
```

PROPOSITION: After DFS on an undirected graph, every edge is ONLY either tree edge or back edge.

Topological sorting

A topological sort (or ordering) of a directed acyclic graph (DAG) is a linear ordering of vertices such that for every directed edge, vertex comes before
.

Method

1. Do a DFS
2. Order vertices in order of DECREASING finishing times f[v]

Strongly Connected Components

Definition. An SCC is a subgraph such that any pair of vertices in the SCC can reach each other.

Computing Strongly Connected Components (Kosaraju)
Practice this!

1. Do DFS on G
2. Order vertices in the reverse order of f[v]
3. Compute G_reverse

4. Do DFS on G_reverse; visiting in the order of (2)

Each separate tree in the forest of 4 is an SCC

Week 10: MST

This week we are concerned with **connected undirected weighted graphs**

G=(V, E) with weights w(e) for all e in Ec

Definitions

Tree: An undirected connected acyclic graph.

It has n-1 edges. Removing any edge would disconnect the tree. Adding an edge would create a cycle.

Basic CSC148 trees would be a special case of this

Minimum Spanning Tree (MST): Consider a connected, undirected, and weighted graph.

An MST would be a tree containing all the vertices in G with the smallest total weight

NOT NECESSARILY UNIQUE

By definition of Spanning , an MST of G would have |V| vertices.

By definition of tree, an MST would thus have |V|-1

Growing algorithms

We want to grow an MST. Essentially, while T is not a spanning tree (|T|<|V|-1), we can add a "safe edge".

A safe edge is one that, when added, assuming T ⊆ some MST, then the new T will ⊆ MST.

The two algorithms are based on this theorem

THEOREM.

Let G be a connected undirected weighted graph, and T be a subgraph of G which is a subset of some MST of G.

Then e is a safe edge if e is the **minimum weighted edge** among all edges of a fixed **connected component** of T.

	How to keep track of connected	Find minimum weight edge

	components	
Prim	Keep one tree. Plus isolated vertices	Use priority queue ADT
Kruskal	Use disjoint set ADT	Sort all edges by weight

Prim's algorithm

We focus on growing one tree and adding one edge at a time.
Specifically,

1. Start from arbitrary vertex as root
2. Among all edges leaving the current tree (edges crossing components), pick the one with the minimum weight.

How do we get that minimum? We store all candidate vertices in a Min Priority Queue.

Component: In this case, either the tree or any isolated vertex. In Kruskal's this may be different.
Component-Crossing Edge: An edge that connects two components.

Algorithm
Start with Min Priority Queue Q which holds all vertices.

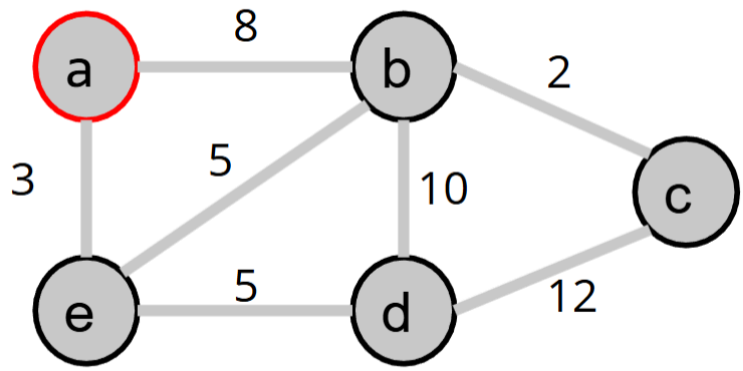
Each vertex is initialized with key=inf and pi=NIL.

Pick a source vertex. Then we consider its neighbours. Set key=weight and pi=[source].

We add the minimum weight neighbour. Then, we update the Q with the newly discovered neighbours.

Now fully updated. We add the minimum weight edge again. Repeat until we have a tree.

a: 0, NIL



Q	key	pi
b	∞ →8	NIL →a
c	∞	NIL
d	∞	NIL
e	∞ →3	NIL →a

Correctness

The added edge is always a safe edge.

Runtime analysis

Assume we use a binary min heap.

In total we have V ExtractMin's.

Each ExtractMin will take O(log V) ⇐ This is characteristic of min heap

In total we check at most O(E) neighbours.

Each neighbour may lead to a DecreaseKey which takes O(log V)

TOTAL: O((V+E) log V) = O(E log V)

since G is a connected graph, E>= V-1. Thus |V| is O(|E|).

Also log|E| is in O(log|V|) because E<=|V|^2.

So log E is at most log V^2 = 2log V.

So the above equation really does boil down to log V

Kruskal's algorithm

Idea

1. Sort all **edges** by weight. Start adding to MST from the lightest one.
    a. Constraint: Added edge must not cause a cycle.
    b. Essentially unioning small trees to big tree

In other words, the two endpoints of the edge must belong to two different trees (components).
So basically,

Take a root. Sort all edges by weight. Take the lightest edge that is not in two components already.

But how do we check different components?

Disjoint sets.

Week 11: Disjoint Sets

Disjoint Sets ADT Definition

The **disjoint sets ADT** stores a collection of nonempty disjoint sets S1, S2, …, Sk.
Each such disjoint set has an associated representative element.

Operations

- MakeSet(x) create set {x}
- FindSet(x) return representative of the set containing x
- Union(x, y) take the two sets that x and y are contained in and merge them.
  the new representative can be one of the two or something else.

Notes

- MakeSet: Requires that x does NOT belong to any set
- Union: If x and y are in the same set nothing happens

Applications: Kruskal's algorithm

We can use it to find connected components of a graph

Recall Kruskal's algorithm: Essentially,

Take a root. Sort all edges by weight. Take the lightest edge that is not in two components already.

But how do we check membership of different components?

Disjoint sets.

Suppose our candidate edge is ei = (ui, vi)

```
if FindSet(ui) != FindSet(vi):
    Union(ui, vi);
    T = T + {ei};
```

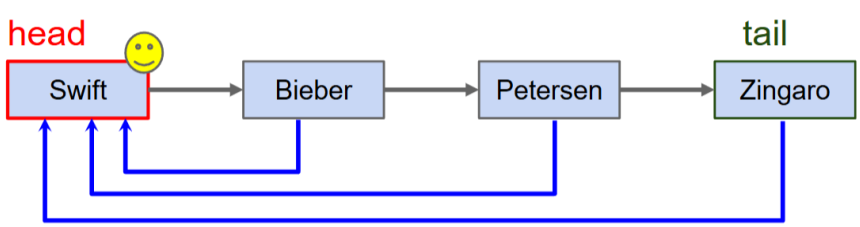
Implementation

Ways of implementations

1. Linked lists with extra pointer and with union-by-weight
2. Trees with union-by-rank
3. Trees with path-compression
4. Trees with union-by-rank and path-compression

Approach 1: LL with pointer

Approach 1: We use a **linked list** augmented with an **extra pointer to the head**.



MakeSet is O(1). FindSet is O(1)

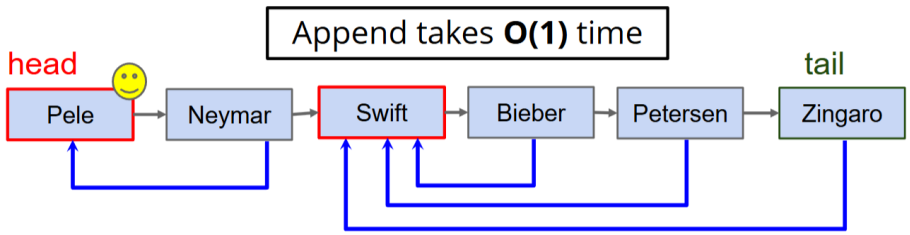
Union…?

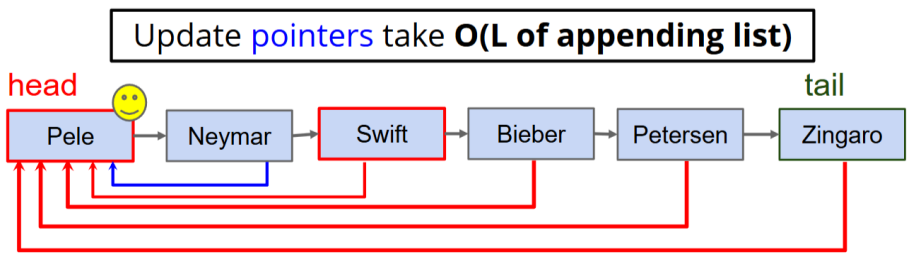
Implementing Union

Idea: append one list to the other then update the pointers of the appended list to point to the new head.

Clearly, updating pointers takes O(Length(appended)). Union is the long one

Linked list with pointer to head (3)





22

Analysis of Approach (Worst-Case Amortized)

Recall: The total cost of a sequence of m operations.

Bad sequence:

- m/2 MakeSet ⇒ m/2 singletons
- m/2 - 1 Union ⇒ construct 1 set of size m/2
- 1 FindSet ⇒ Get representative of just 1

Where in Union, we always append the larger set to the smaller set.

Total cost dominated by the Union.

$\Theta(1 + 2 + \ldots + m/2\text{-}1) = \mathbf{\Theta(m^2)}$ ← (Sum formula)

Approach 2: Linked list with extra pointer to head with **union-by-weight**.

Let's be smart. Instead, in Union, append the shorter to the longer one.

We track the size (weight) of each list. This is an extra augmentation.

CLAIM: For a sequence with m operations including n MakeSet, total cost is

O(m + n log n)

In the previous scenario with m operations, it was $O(m^2)$.

Now it would be O(m+(m/2) log (m/2)). Obviously dominated by

O(m log m)

Much better than quadratic.

PROOF OF CLAIM:

- Let x be an element. How many times must its head pointer be updated?
- Union-by-weight means when x is updated, it is in the smaller list of two. Thus after union, the size of the list doubles.
    - Every time x is updated, set size doubles.
- There are only n elements in total so we double at most O(log n) times.
- True for all n elements. So total updates O(n log n)

Approach 2: Trees

Well actually there are 3 variants. union-by-rank, path-compression, and both.

Concepts

We consider making each set as an "inverted" tree. That is, each element has a pointer to its parent. The root points to itself.

MakeSet: O(1) ← create single node tree with root x
FindSet: O(h) ← Trace up the parent pointer until root is reached.
Union???

Implementing Union (naive)

We can do some nice things with pointers. See this approach to Union(Bieber, Buffet)

1. O(h) Call FindSet(x) and FindSet(y) to locate representatives.
2. O(1) Point one tree's root to the other tree's root.

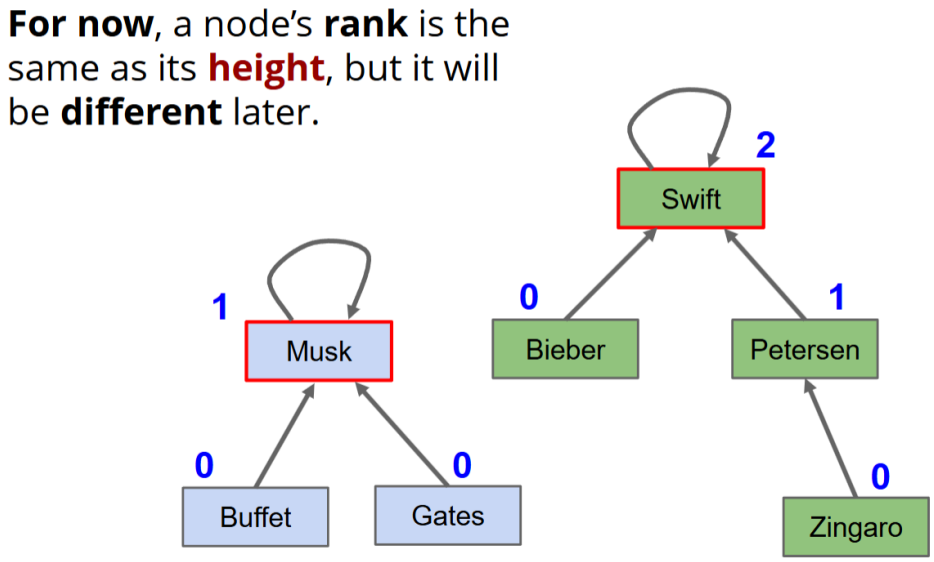
Runtime: In the worst case, m operations.
Worst case sequence: m/4 MakeSet, m/4 - 1 Union, m/2 + 1 FindSet
Total cost in WSQ: **$\Theta(m^2)$**

Union-by-rank

Since FindSet takes O(h), height of the tree matters. To keep height small, the smaller tree should take the taller tree's root. But how do we keep track of the height?

Solution: Each node keeps a rank. ATP rank is the same as height. Leaf has 0 and root has h.

Upon **union**, let the root with the lower rank point to the higher rank root.
If same rank, choose one to point to the other and increment rank.



Runtime with union by rank

Proposition: Rank of the root of a tree with n nodes is at most log n, i.e., r(n) <= log n

Prove this by induction. Equivalent to proving n(r) >= 2^r

Thus, a tree of n nodes formed this way has height log n.

Therefore FindSet takes O(h)=O(log n)

O(log n). MakeSet takes O(1)

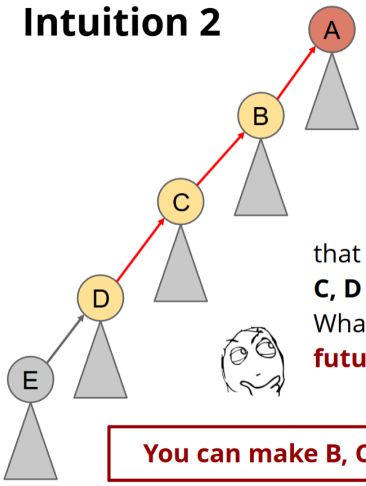
Union takes the time of 2 FindSet (find the representative) and O(1) so total O(log n)

The total cost of m operations is thus **O(m log n)**.

Trees with Path Compression

Intuition

Intuition 2



Now I do a
FindSet(D)

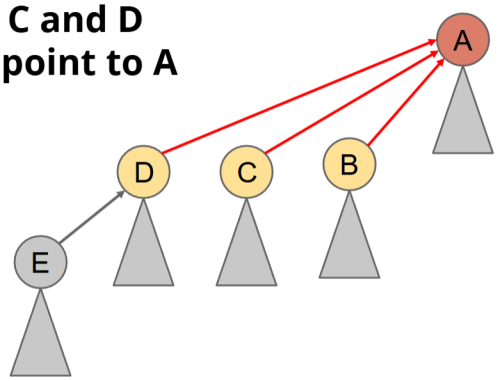
On the way of finding **A**, you visit **D**, **C**, **B** and **A**.

that is, now you have access to **B**, **C, D** and the root **A**.
What **nice** things can you do for **future FindSet** operations?

You can make B, C and D super close to A!

47

Make B, C and D directly point to A



In other words, the path D→C→B→A is "compressed".

Extra cost to FindSet: at most **twice** the cost, so does not affect the order of complexity

48

how does this work?

Runtime: Can be proven: for any sequence of m operations with n elements, the worst-case total cost of the sequence is O(m log n)

Summary of the 3 ways

Our benchmark has been the worst-case total cost of a sequence of m operations.
For LL with pointer, Θ(m log m). For Trees with Union by Rank, Θ(m log m). With path compression, Θ(m log m).

We can do better: union by rank PLUS path compression!

Approach 3: Union by Rank + Path Compression

Path compression happens in FindSet. Union by rank happens in Union. They don't interfere! Just use both.

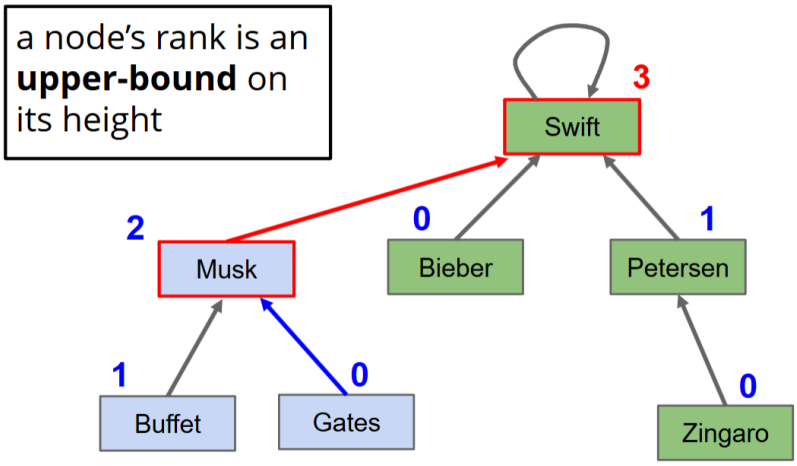
Pseudocodes summary

```
MakeSet(x):
    x.p = x
    x.rank = 0

FindSet(x):
    if x != x.p
        x.p = FindSet(x.p)
    return x.p

Union(x,y):
    Link(FindSet(x), FindSet(y))

Link(x,y):
    if x.rank > y.rank:
        y.p = x
    else:
        x.p = y
        if x.rank == y.rank:
            y.rank += 1
```

Exercise: Draw result

Rank vs. Height

Path compression does not maintain height.
A node's rank is an upper bound on its height.



Rank

Runtime of this.
O(m * a(n)) where a is the inverse Ackermann function. This is essentially constant.
Now the total cost of our sequence is roughly **O(m)**

Conclusion: This is a remarkable improvement from the previous attempts!

Week 12: Lower Bounds?

Note: we start with a review.

We usually talk about upper bonds on algorithmic complexity. But analyzing lower bounds can also help us understand our limits.

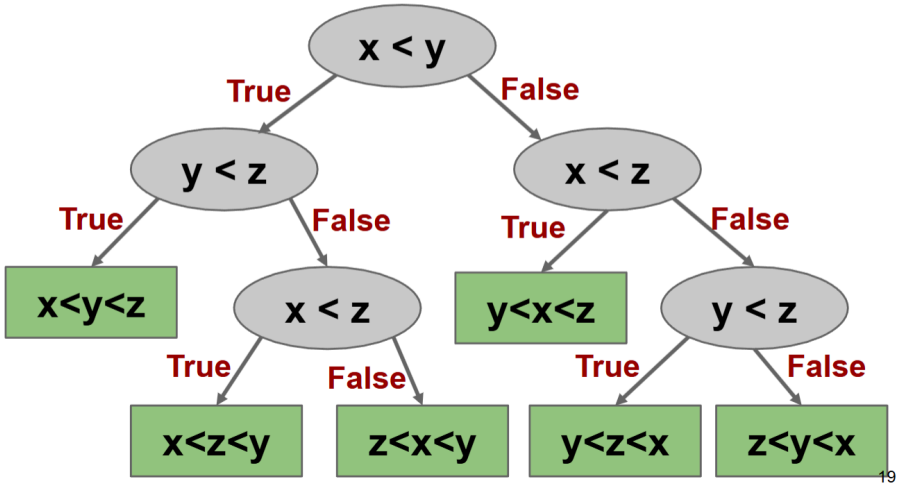
Proving lower bound of comparison-based sorting algorithms

Assumptions: No assumptions. Comparison based.

Suppose we want to sort {x, y, z}. Assume they are distinct values.
How do we order them? We can write this as a decision tree

The decision tree for sorting {x, y, z}

A tree that contains a complete set of decision sequences.
Every comparison-based sorting algorithm has a corresponding decision tree.



Each leaf node is a possible sorted order.

How many possible orders for n elements? n!

Therefore the **number of leaves** L >= n!

Now think about the **height** of the tree.

Recall: A binary tree with height h as at most 2^h leaves.

So **number of leaves** L <= 2^h.

We have the inequality 2^h ≥ L ≥ n!

⇒ h >= log(n!)

Rearrange to get

>= log 1 + log 2 + … + log n/2 + … + log n

>= log n/2 + … + log n

>= log n/2 + … + log n/2

>= (n/2) log n/2

Hence h is in Ω(n log n). Since h represents the worst-case number of comparisons to sort (binary tree), we just showed the worst-case number of comparisons is Ω(n log n). QED

Similar problem

IRL: How many matches need to be played to determine a champion out of 10 teams?
Each match eliminates at most 1 team. Need to eliminate 9 teams in order to determine a champion.

Generalized: Given n elements, determine the maximum element.
How many comparisons are needed at least?

→ Need at least n-1 comparisons

Designing an algorithm to approach the limit
How do we design an algorithm that meets this?